Dokumentation: SDM120 Modbus Energiemonitor mit ESP32

Diese Dokumentation fasst die vollständige Implementierung zur Erfassung des Stromverbrauchs einer einphasigen Wohnung (230V AC, max. 25A) mittels eines **Eastron SDM120-Modbus** und eines **ESP32-WROOM** im autarken **Access-Point-Modus** zusammen.

---

1. Hardware-Spezifikationen & Anschluss

Da die Anlage mit maximal 25 A abgesichert ist, wird die Variante des **SDM120-Modbus für Direktmessung (bis 45 A)** genutzt. Ein externer Stromwandler (CT) ist nicht erforderlich.

Klemmenbelegung SDM120

SDM120 Klemme	Signal / Beschreibung	Anschlussleitung / Ziel
Klemme 1	Line In (Phase Eingang)	Hauptsicherung / Zählerabgang
Klemme 2	Line Out (Phase Ausgang)	Abgang zur Unterverteilung / FI-Schutzschalter
Klemme 3	Neutralleiter (N)	N-Sammelschiene
Klemme 8	RS485 Ground (GND)	GND des Pegelwandlers (Optional zur Potentialisierung)
Klemme 9	RS485 B (Invertiert)	Pin **B** des RS485-TTL-Wandlers
Klemme 10	RS485 A (Nicht-Invertiert)	Pin **A** des RS485-TTL-Wandlers

Verdrahtung zum ESP32 (über RS485-TTL-Wandler)

Da der ESP32 mit **3,3V TTL-Pegeln** arbeitet, muss ein Pegelwandler (z. B. MAX485 oder ein Modul mit automatischer Flussrichtungssteuerung) dazwischengeschaltet werden.

- **RO (Receiver Output):** an **GPIO 16 (RX2)** des ESP32
- **DI (Driver Input):** an **GPIO 17 (TX2)** des ESP32
- **DE / RE (Data/Receiver Enable):** gebrückt an **GPIO 4** des ESP32 (Sende-/Empfangsumschaltung)
- **VCC / GND:** Passende Spannungsversorgung des Wandlers (Achtung: ESP32 Pins sind nicht 5V-tolerant!)

---

2. Modbus-Register (Function Code 04 - Input Registers)

Die Messwerte liegen als **32-Bit-IEEE-754-Floating-Point-Zahlen** vor (Big-Endian, High-Word zuerst):

- **0x0000 (30001):** Spannung [V]
- **0x0006 (30007):** Stromstärke [A]
- **0x000C (30013):** Wirkleistung [W]
- **0x0142 (30323):** Gesamt-Wirkenergie [kWh]

Hinweis: Stellen Sie den SDM120 im internen Setup-Menü (Langes Drücken der Taste -> `-SET-`) idealerweise auf **9600 Baud** und Modbus-ID **1** um.

---

3. Vollständiger Programmcode (Arduino IDE)

Benötigte Bibliotheken

Installieren Sie folgende Bibliotheken über den Bibliotheksverwalter der Arduino IDE:

1. **ModbusMaster** (von Doc Walker)
2. **ESPAsyncWebServer** (von Me-No-Dev / mathieucarbou)
3. **AsyncTCP** (von Me-No-Dev / mathieucarbou)

```
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include <ModbusMaster.h>

// Gewünschte Zugangsdaten für das ESP32-WLAN (Access Point)
const char* ap_ssid     = "SDM120_Zaehler_AP";
const char* ap_password = "Kombischraube2026!"; // Mindestens 8 Zeichen!

// Feste IP-Konfiguration für den Access Point
IPAddress local_IP(192, 168, 4, 1);
IPAddress gateway(192, 168, 4, 1);
IPAddress subnet(255, 255, 255, 0);

// Hardware-Pins für den RS485-Pegelwandler (HardwareSerial 2)
#define RX_PIN 16
#define TX_PIN 17
#define DE_RE_PIN 4  // HIGH = Senden, LOW = Empfangen

// Globale Messvariablen
float voltage = 0.0;
float current = 0.0;
float power   = 0.0;
float energy  = 0.0;

ModbusMaster node;
AsyncWebServer server(80);

// RS485 Sende-/Empfangsumschaltung (Pre- und Post-Transmission)
void preTransmission() {
  digitalWrite(DE_RE_PIN, HIGH);
}
void postTransmission() {
```

```
  digitalWrite(DE_RE_PIN, LOW);
}

// Hilfsfunktion: Wandelt zwei 16-Bit Modbus-Register in ein 32-Bit Float (IEEE
754)
float regToFloat(uint16_t reg1, uint16_t reg2) {
  uint32_t combined = ((uint32_t)reg1 << 16) | reg2;
  float floatVal;
  memcpy(&floatVal, &combined, sizeof(float));
  return floatVal;
}

// Liest die Daten vom SDM120 (Modbus-Adresse 1, Function Code 04)
void readSDM120() {
  uint8_t result;

  // 1. Spannung abfragen (Register 0x0000, Länge 2)
  result = node.readInputRegisters(0x0000, 2);
  if (result == node.ku8MBSuccess) {
    voltage = regToFloat(node.getResponseBuffer(0), node.getResponseBuffer(1));
  }

  // 2. Stromstärke abfragen (Register 0x0006, Länge 2)
  result = node.readInputRegisters(0x0006, 2);
  if (result == node.ku8MBSuccess) {
    current = regToFloat(node.getResponseBuffer(0), node.getResponseBuffer(1));
  }

  // 3. Wirkleistung abfragen (Register 0x000C, Länge 2)
  result = node.readInputRegisters(0x000C, 2);
  if (result == node.ku8MBSuccess) {
    power = regToFloat(node.getResponseBuffer(0), node.getResponseBuffer(1));
  }

  // 4. Gesamt-Wirkenergie abfragen (Register 0x0142, Länge 2)
  result = node.readInputRegisters(0x0142, 2);
  if (result == node.ku8MBSuccess) {
    energy = regToFloat(node.getResponseBuffer(0), node.getResponseBuffer(1));
  }
}

// HTML & JavaScript Oberfläche (wird im Flash-Speicher abgelegt)
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="de">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Wohnung Stromzähler</title>
    <style>
        body { font-family: Arial, sans-serif; background-color: #f4f6f9;
color: #333; margin: 0; padding: 20px; display: flex; justify-content: center;
}
```

```
        .container { background: white; padding: 30px; border-radius: 12px;
box-shadow: 0 4px 15px rgba(0,0,0,0.1); max-width: 400px; width: 100%; }
        h1 { font-size: 24px; text-align: center; color: #0056b3; margin-
bottom: 25px; }
        .data-box { display: flex; justify-content: space-between; padding:
15px 0; border-bottom: 1px solid #eee; font-size: 18px; }
        .data-box:last-child { border-bottom: none; }
        .label { font-weight: bold; color: #555; }
        .value { color: #222; font-variant-numeric: tabular-nums; }
        .unit { font-size: 14px; color: #777; margin-left: 4px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>SDM120 Energiemonitor</h1>
        <div class="data-box"><span class="label">Spannung:</span><span><span
class="value" id="val_v">--.-</span><span class="unit">V</span></span></div>
        <div class="data-box"><span class="label">Stromstärke:</span><span>
<span class="value" id="val_a">--.-</span><span class="unit">A</span></span>
</div>
        <div class="data-box"><span class="label">Leistung:</span><span><span
class="value" id="val_w">--.-</span><span class="unit">W</span></span></div>
        <div class="data-box"><span class="label">Gesamtverbrauch:</span><span>
<span class="value" id="val_kwh">--.-</span><span class="unit">kWh</span>
</span></div>
    </div>
    <script>
        function updateData() {
            fetch('/api/data')
                .then(response => response.json())
                .then(data => {
                    document.getElementById('val_v').innerText =
data.voltage.toFixed(1);
                    document.getElementById('val_a').innerText =
data.current.toFixed(2);
                    document.getElementById('val_w').innerText =
data.power.toFixed(0);
                    document.getElementById('val_kwh').innerText =
data.energy.toFixed(2);
                })
                .catch(err => console.error('Fehler beim Datenabruf:', err));
        }
        setInterval(updateData, 2000); // Alle 2 Sekunden aktualisieren
        updateData(); // Erstaufruf
    </script>
</body>
</html>
)rawliteral";

void setup() {
  Serial.begin(115200);

  // RS485 Richtungs-Pin konfigurieren
```

```
  pinMode(DE_RE_PIN, OUTPUT);
  digitalWrite(DE_RE_PIN, LOW); // Start im Empfangsmodus

  // HardwareSerial 2 für Modbus (9600 Baud, 8N1)
  Serial2.begin(9600, SERIAL_8N1, RX_PIN, TX_PIN);

  // Modbus Knoten-ID konfigurieren (Standard beim SDM120 ist 1)
  node.begin(1, Serial2);
  node.preTransmission(preTransmission);
  node.postTransmission(postTransmission);

  // WLAN in den Access-Point-Modus versetzen
  WiFi.mode(WIFI_AP);
  WiFi.softAPConfig(local_IP, gateway, subnet);

  if (WiFi.softAP(ap_ssid, ap_password)) {
    Serial.println("\nAccess Point erfolgreich gestartet!");
    Serial.print("WLAN-Name (SSID): ");
    Serial.println(ap_ssid);
    Serial.print("IP-Adresse für den Browser: ");
    Serial.println(WiFi.softAPIP());
  } else {
    Serial.println("AP-Start fehlgeschlagen!");
  }

  // Webserver Routen definieren
  // 1. Hauptseite ausliefern
  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

  // 2. JSON API Endpunkt für das JavaScript des Browsers
  server.on("/api/data", HTTP_GET, [](AsyncWebServerRequest *request){
    String json = "{";
    json += "\"voltage\":" + String(voltage, 2) + ",";
    json += "\"current\":" + String(current, 3) + ",";
    json += "\"power\":" + String(power, 1) + ",";
    json += "\"energy\":" + String(energy, 2);
    json += "}";
    request->send(200, "application/json", json);
  });

  server.begin();
}

void loop() {
  static unsigned long lastMillis = 0;

  // Alle 2 Sekunden den Zähler über Modbus auslesen
  if (millis() - lastMillis > 2000) {
    lastMillis = millis();
    readSDM120();
```

```
    }
}
```

---

4. Inbetriebnahme im Browser

1. Suchen Sie an Ihrem Endgerät nach dem WLAN `SDM120_Zaehler_AP` und verbinden Sie sich mit dem Passwort `Kombischraube2026!`.
2. Falls Ihr Betriebssystem meldet, dass kein Internet vorhanden ist, wählen Sie **„Trotzdem verbunden bleiben“**.
3. Öffnen Sie einen Webbrowser und rufen Sie die Adresse `http://192.168.4.1` auf.